

Lesson 12: Distributed Databases

What is a Distributed Database?

- A **Distributed DBMS (DDBMS)** allow for the storage and processing of data distributed across multiple site (two or more)
- The processing and storage can be decoupled from one another
 - Distributed processing allows for multiple sites to connect to one databases to write or read data
 - A distributed database is a logically related database stored across multiple sites, it requires distributed processing
 - Stored as **database fragments** or a subset of a database at different sites
- In both cases:
 - Sites are independent of one another
 - Sites are networked together

What came before?

- Less organizations had a global presence
- The internet didn't exist or wasn't as prevalent
- Centralized, single site DBMS
 - Think what we have set up in lab, but hosted on a server in a server room

Why are they needed?

- More organizations have a global presence
 - Including customers and employees
- The internet has changed how business is done
 - Consumers expect fast and on-demand transactions
 - Reach can easily be global
- Mobile devices amplified the demands that already became the norm of the internet
- Many organizations shifted to using Big Data

Advantages Centralized vs. Distributed

Centralized	Distributed
Less complex	Reduction of single point of failure
Easier to implement security	Horizontally scalable (add more machines)
Lower costs in terms of people and software	Faster data access and processing
Limitations in the volume and velocity of data	Storage and processing are decoupled

Disadvantages Centralized vs. Distributed

Centralized	Distributed
Only vertically scalable (add more CPU/RAM/HD)	More complex management
Processing and storage on same server	More difficult to implement security well
Costs: physical infrastructure	Costs: platform, people and software
Processing and storage unlikely decoupled	Big Data can thrive

DDBMS Components

- Sites or Nodes form the network of interconnected machines
- Network Hardware/Software enable the nodes to communicate
- Communication media carry the data between sites/nodes
- Transaction processor requests the data
- Data Processor stores and retrieves the data

Data and Process Distribution Levels

- Single-site Processor and Data (SPSD) is much like our lab environment data is stored and processed on the same system
- Multi-site Processing and Single Site Data (MPSD) has multiple processes run on different workstations and all the data stored in a single system
 - Uses client-server architecture
- Multi-site Processing and Data (MPMD) is a fully distributed DBMS enabling the processing and storage of data to occur across multiple sites/nodes

Distributed Database Features

- Just like a DBMS, many of the complexities are abstracted away and hidden from the end users
- There are extra things to hide from end-users in a distributed database

DDBMS Transparency Features

- Distribution Transparency allow the database to be treated as a single logic database
- Transaction Transparency allows multi-site data operations
- Failure Transparency ensures continued operation even in the event of the failure at one site/node
- Performance Transparency mimics the performance of a centralized DBMS
- Heterogeneity Transparency enables the integration of multiple DBMSs under a single schema

Distribution Transparency

Level of transparency	User Needs Fragment Name	User Needs Fragment Location
Fragmentation	No	No
Location	Yes	No
Local Mapping	Yes	Yes

Transaction Transparency

Level of transparency	Number of requests	Number of sites/nodes
Remote Request	1	1
Remote Transaction	≥ 1	1
Distributed Request	1	> 1
Distributed Transaction	≥ 1	> 1

Two Phase Commit Protocol

- Similar to atomicity, if any part of a transaction fails, it's aborted and rolled back
- In a distributed environment, transactions could be taking place using data across multiple, so the final data is not committed until it's successful at all sites
- The two-phase commit protocol (2PC) is used to ensure consistency
- The 2PC protocol uses the write-ahead protocol and the do-undo-redo protocol
 - Write the transactions to the log before the database
- The 2PC protocol defines a coordinator node/site and one or more subordinates
 - How the coordinator node is selected varies by DDBMS

Performance/Failure Transparency

- Distributed databases emphasize high availability, meaning the wait time between a request and response is minimized
- The goal is to replicate the speed of a centralized database, even though we're doing so over a much larger (potentially geographically distant) network of sites/nodes
- Unlike a centralized database, the transaction processor has to determine which fragments of data to access
- The consistency of the database replicas also need to be maintained
- The network and node availability or response can't be predetermined, but the network latency should be considered

Distributed Database Design

Fragmentation (What)

- **Horizontal Fragmentation** – breaks apart into a subset of unique rows which are stored on different node
- **Vertical Fragmentation** – breaks apart data into subset of unique columns which are stored on different nodes
- **Mixed Fragmentation** – a combination of both vertical and horizontal fragmentation

Data Replication

- The storage of data copies at multiple sites/nodes
- Subject to the mutual consistency rule that requires all copies of data fragments to be identical

Data Replication Propagation

- In **push replication** the originating node sends the changes to the replica nodes to update the data ASAP. This decreases availability, but increases consistency.
- In **pull replication** the originating nodes sends messages notifying the replica nodes of the changes and allowing them to make them at their own speed. This increases data availability, but decreases consistency at a point in time.

Data Replication (How)

- A **fully replicated database** stores multiple copies of each fragment at different sites
- A **partially replicated** database stores multiple copies of some database fragments at multiple sites
- An **unreplicated database** stores each fragment at a single site
- Costs, size and usage should dictate which method of replication best suits and organization

Data Allocation (Where)

- A **centralized data allocation** the entire database is stored at one site
- A **partitioned data allocation** the database is divided into two or more disjointed parts and stored at two or more sites
- A **replicated data allocation** copies of one or more fragments are stored at several sites

The CAP Theorem

- This theorem describes the most desirable properties of a distributed data system
- Consistency ensures that the data stored across sites/nodes is the same at any given point in time
- Availability ensures that all requests are fulfilled and none are lost
- Partition Tolerance ensures continued operation even with a sites/nodes failure

ACID vs. BASE

- Data consistency within a distributed databases is more complex and may require some trade-offs
- Enter Basically Available, Soft state, Eventually consistent (BASE)
- BASE relies on the assumption that the data will eventually become consistent

Homework

- Read Chapter 13